Rayhan Ahmed Shium
Prof. Matthew McGeehan
Cybersecurity Fundamentals
October 9, 2026

Hacker's Toolkit & User Handbook

Introduction

This toolkit contains 12 free or community tools that are used for authorized security assessment and provide the following: Reconnaissance, Network Mapping, Vulnerability Detection, Penetration Testing, and Post-assessment Investigation. Test these tools on systems that you own or are given permission to test.

You will find a tool inventory, a comparison of Nmap, OpenVAS (Greenbone Community Edition), and Nikto, and a practical Nmap handbook in Section 3. References are in APA style at the end.

Section 1: Toolkit Inventory

There are 12 tools in 10 categories in the toolkit. The table provides a quick view, while each tool is then provided with a short description of its purpose, usage, pros and cons.

#	Tool	Category	Platforms	Cost / License	Source
1	Nmap	Reconnaissance, Network Analysis	Windows, macOS, Linux, BSD	Free, open source	nmap.org
2	Maltego Community Edition	Reconnaissance (OSINT)	Windows, macOS, Linux	Free with registration	maltego.com
3	Wireshark	Network Analysis	Windows, macOS, Linux	Free, open source (GPL)	wireshark.org
4	OpenVAS / Greenbone Community Edition	Vulnerability Assessment	Linux (packages, source, containers)	Free, open source	greenbone.net
5	Nikto	Web Application Testing, Vulnerability Assessment	Any OS with Perl	Free, open source	github.com/sullo/nikto
6	Burp Suite Community Edition	Web Application Testing	Windows, macOS, Linux	Free (Pro is paid)	portswigger.net
7	John the Ripper	Password Auditing	Windows, macOS, Linux	Free, open source	openwall.com
8	Aircrack-ng	Wireless Security	Linux (best), limited Windows/macOS	Free, open source	aircrack-ng.org
9	Metasploit Framework	Exploitation, Penetration Testing	Linux, macOS, Windows	Free, open source (Pro is paid)	metasploit.com

#	Tool	Category	Platforms	Cost / License	Source
10	Autopsy	Digital Forensics	Windows (primary), Linux, macOS	Free, open source	autopsy.com
11	Ghidra	Malware Analysis, Reverse Engineering	Windows, macOS, Linux (Java)	Free, open source	ghidra-sre.org
12	Security Onion	Log Analysis, Incident Response, Network Monitoring	Its own Linux-based distribution	Free, open source	securityonion.net

1. Nmap (Network Mapper)

Primary purpose: Finds live hosts on a network and their open ports, services, service versions and operating systems.

Common scenarios to use: Create an asset inventory, post-change firewall configuration review, discovery of unauthorized services, and initial scan for any penetration test.

Strengths: Quick, programmable and extremely precise. It's enhanced with hundreds of scripts from the Nmap Scripting Engine to perform more in-depth inspections. Can save output in text, XML or grepable format for other tools.

Limitations: It reveals what's exposed, not necessarily if it is exploitable in depth. The "aggressive" scans are noisy and will be detected by intrusion detection systems. There are types of scan that require administrator or root access.

2. Maltego Community Edition

Primary purpose: Collects open-source intelligence (OSINT) and draws the links between people, domains, email addresses, IP addresses and organizations as a graph.

Typical use cases: Mapping an organization's internet footprint before a test, investigating phishing infrastructure, and threat-intelligence research.

Strengths: The graph view makes relationships easy to see. "Transforms" automate lookups that would take hours by hand.

Restrictions: Free version restricts number of results returned by each transform and requires an account. There are a lot of helpful sources of data that are paid add-on items.

3. Wireshark

Primary function: Can capture network traffic and decode it packet by packet through hundreds of protocols.

Common applications: Network troubleshooting, identification of clear text credentials, malware traffic analysis, and protocol analysis.

Strengths: Powerful filters for both captured and live packets, the ability to extract files from captures, and the ability to reassemble TCP streams.

Limitations: It is difficult to do large captures by hand. Without the session keys, little can be seen from encrypted traffic (TLS). It only sees traffic if it reaches the capturing interface.

4. OpenVAS / Greenbone Community Edition

Primary purpose: Executes authenticated and unauthenticated vulnerability scans using a feed of tens of thousands of vulnerability tests.

Common scenarios: Regular vulnerability scanning, identifying missing patches, generating compliance-based reports on a network.

Strengths: The scanner is comprehensive and free, with a web interface, scheduling, and severity ratings linked with CVEs.

Limitations: It is the most difficult tool in this toolkit to install. First feed sync is very slow, and the scanner requires more RAM and disk than the other tools. The community feed is lagging behind the commercial feed and false positives have to be checked manually.

5. Nikto

Primary purpose: Searches web servers for dangerous files, old server software, default files and typical misconfigurations.

Common scenarios: Performing a preliminary analysis of a web server, discovering unprotected admin pages or backup files, and verifying the existence of server headers and versions.

Strengths: It is easy to use, small in size, and quick to learn.

Limitations: Very noisy, with no stealth. It scans for known issues on the server, not logic problems within a custom web app and it creates false-positive reports.

6. Burp Suite Community Edition

Primary purpose: Intercepts the requests between the browser and the web application and allows the tester to view and alter all requests.

Common scenarios: Manual testing of web applications, changing of parameters, login and session management testing, and replaying requests with Repeater.

Strengths: It is the common tool used for doing manual web testing. Most daily needs can be found in Proxy, Repeater and Decoder.

Limitations: The free edition doesn't include an automated vulnerability scanner and Intruder is limited by rate. The project files are not saved between sessions.

7. John the Ripper

Primary purpose: Crack the password hashes to check the strength of the password.

Common scenarios: Do an audit of an organization's password policy using authorized hash dumps, gain access during a test, or use it in a CTF exercise.

Strengths: Supports many hash formats (especially the "jumbo" build) and auto-detects many of them. The wordlist, rule and incremental modes are flexible.

Limitations: CPU cracking takes more time than GPU based cracking tools like Hashcat. It's possible that strong, long passwords can never be broken in a useful time.

8. Aircrack-ng

Primary purpose: Suite to audit Wi-Fi: capture traffic, capture WPA/WPA2 handshakes and test the strength of the passphrase.

Typical use cases: Checking that a wireless network uses a strong passphrase, finding rogue access points, and wireless security labs.

Pros: All the tools to test 802.11 in one package, well-documented.

Restrictions: Must be running on a wireless adapter capable of monitor mode and packet injection. This is best suited for Linux. Cracking only works if the passphrase is weak enough to appear in a wordlist.

9. Metasploit Framework

Primary purpose: Includes exploits, payloads and post-exploitation modules to verify that it is possible to exploit a vulnerability.

Use cases: Lab usage like Metasploitable; demonstrating actual risk to management; validation of scanner results.

Areas of Strength: Large module library, Meterpreter payload, and importing Nmap results into its own database for integration of results.

Restrictions: Can crash services or damage systems when not used properly. It is detected by anti-virus software. Running modules without understanding them teaches very little.

10. Autopsy

Primary purpose: A graphical digital forensics platform designed around The Sleuth Kit that can be used to analyze disk images and phones.

Commonly used applications include: Recovery of deleted files, creation of a history of user activity, searching for keywords and reviewing web usage during investigation.

Pros: It's free and easy to use, and includes plug-in modules for hashing, detecting file types, timelines and keyword search.

Limitations: Not very fast for very large images. Commercial suites have additional device types supported. Even with the evidence, the examiner has to adhere to proper procedure.

11. Ghidra

Primary purpose: A set of tools created by the U.S. National Security Agency for the purpose of reverse engineering compiled programs.

Common applications: Static malware analysis, analysis of an interesting binary or vulnerability research.

Strengths: Completely free, with a decompiler for many processor types and scripting in Java and Python.

Limitations: Steep learning curve, and it requires knowledge of assembly. Malware samples should be handled within a Virtual Machine (VM) that is isolated.

12. Security Onion

Primary purpose: A Linux distribution that packages network security monitoring, intrusion detection, log management and case management into a suite.

Common use cases: Small SOCs, threat hunting, analyzing alerts, and incident response investigations.

Strengths: Allows for a single install of tools like Suricata, Zeek and Elastic-based search with dashboards pre-installed.

Restrictions: Requires a significant amount of RAM (16 GB or more) and a dedicated machine or VM. Real time is needed to set up alerts and familiarize yourself with the platform.

Section 2. Tool Comparison: Nmap, OpenVAS and Nikto

All three of these can be used to perform authorized security assessment, but offer different answers to the questions asked: Nmap can say what hosts and services are available on a network and which ports are reachable; OpenVAS can tell if any known vulnerabilities exist on those machines; Nikto can tell you what is exposed on a web server and how it is configured.

Criterion	Nmap	OpenVAS / Greenbone	Nikto
Primary focus	Hosts, ports, services	Known vulnerabilities and risk	Web-server weaknesses
Coverage	Network / TCP and UDP	Hosts, services and software	HTTP / HTTPS servers
Setup	Low	High; feeds and services	Low; Perl
Output	Text / XML	Severity-ranked reports	Web findings / reports
Best use	Discovery and validation	Periodic vulnerability management	Targeted web checks
Key limitation	No comprehensive risk rating	Slow and resource-heavy	No custom application-logic testing

Similarities and differences

Each tool tests systems with probes and generates evidence for a security report. Results need to be validated, as missed results and false positives may occur because of filtering, configuration differences and signatures. Nmap's goal is breadth and speed, OpenVAS' goal is known vulnerabilities, and Nikto concentrates on common web-server exposures.

Nmap generates output which can be compared, in the form of port-state tables and XML for inventories; OpenVAS generates findings with severity and remediation context; Nikto provides output for web servers with observed files and insecure headers. These evidence types are not interchangeable; they complement each other (Lyon, 2009; Greenbone, n.d.; Sullo, n.d.).

Advantages, disadvantages and appropriate situations

Nmap is possibly the most lightweight and flexible of the tools, useful for inventory, firewall checks and service discovery, but open ports do not necessarily indicate vulnerability. Use it first when the scope and active services are unknown.

OpenVAS offers vulnerability IDs, severity and remediation information, including credentialed assessment. It requires feed maintenance, processing time and careful review of results. Use it for regular server audits and to set patching priorities.

Nikto is fast and can identify common misconfigurations of web servers and exposed files. It is noisy, can report low-value findings and cannot replace manual testing of websites. Use it after discovering an authorized HTTP service.

Selection example: If an administrator needs to check whether a firewall change blocked SSH, that is an immediate question that can be answered with an Nmap port check. A security team that has to do a monthly patch report should use Greenbone, due to its severity and remediation information; this allows them to prioritize patching. A developer checking a staging server should use Nikto to identify exposed files, and then manually verify the findings. These tools supplement each other and are not in competition.

Decision criteria: If speed, breadth, and low resource use are important, use Nmap. If vulnerability coverage, repeatable scanning and remediation reporting are important, use OpenVAS. If the authorized target is an HTTP service, use Nikto. If any of the tools reports a suspicious positive result, it should be investigated for false positives, contextual risk and compensating controls.

Section 3. Nmap User Handbook

3.1 Introduction

Nmap (Network Mapper) is a free open source network scanner that includes a host discovery utility for finding hosts, open ports and network services. This guide is suitable for cybersecurity students, junior cyber analysts and system administrators who have basic knowledge of IP addressing and TCP/UDP. Some of the more common uses include asset inventories, attack surface reviews, and verification of firewalls and reconnaissance in an approved assessment. Learning outcomes: the reader should be able to validate an installation, locate authorized hosts, identify available TCP services, gather version information, record evidence and discuss uncertainty in findings. Nmap will not replace a vulnerability assessment or manual validation (Lyon, 2009).

3.2 Installation and initial configuration

Requirements: Windows, Linux or macOS, a terminal, a network connection and administrator privileges for certain scan modes. Simple scans can be performed with a small computer or virtual machine. Download the latest version from nmap.org/download.html.

Platform	Installation / verification
Kali / Debian / Ubuntu	sudo apt update; sudo apt install nmap -y; nmap --version
Windows	Run the official Nmap installer, install Npcap when prompted, then run nmap --version in Command Prompt.
macOS	Install the official package or use brew install nmap; verify with nmap --version.

Prior to scanning: check the list of allowed target addresses; find your local IP using either ip -br a (Linux) or ipconfig (Windows); make a directory to store the results in, using mkdir -p scans (Linux); and update the NSE script index file with sudo nmap --script-updatedb if necessary.

The host operating system for the demonstration was Windows 11. A Kali Linux virtual machine (10.0.2.15) ran in VirtualBox NAT mode on a local 10.0.2.0/24 lab network. The scanned targets were this lab network and scanme.nmap.org, a host the Nmap Project provides for practice scans. The screenshots below are from that lab; outputs may differ depending on the network and test time.

Installation for Debian, Ubuntu or Kali: run ‘sudo apt update’ and afterwards ‘sudo apt install nmap’. On Windows, run the official installer and follow the instructions, including the Npcap install if prompted. On macOS, use the official package or ‘brew install nmap’ (if Homebrew is installed). Use the command “nmap --version” to check each installation (Nmap Project, n.d.-a).

```
nmap --version
```



Figure 1. Nmap version output in my Kali VM.

3.3 Usage guide: four core functions

Function 1. Discover live hosts

Step 1: Check that the subnet is your authorized lab network.

Step 2: Open a terminal and enter:

```
nmap -sn 10.0.2.0/24
```

Step 3: Check the reported live IP addresses. The -sn option does host discovery without doing a port scan. Result: the host addresses and a summary of the number of responses received. Firewalls may drop responses.



Figure 2. Host-discovery output from my lab.

The expected output is a "Nmap scan report for 10.0.2.x" line for each host that responds, followed by a total like ‘256 IP addresses (3 hosts up)’. Discovery might not detect hosts that are blocking probes, and the lack of a response does not mean that a host address is not in use.

Function 2. Check open TCP ports

Step 1: Select an authorized target. In this lab I used scanme.nmap.org, which the Nmap Project allows for light practice scans.

Step 2: Perform a TCP connect scan of the common ports:

```
nmap -sT -T4 -F scanme.nmap.org
```

Step 3: Read PORT, STATE and SERVICE. Open means that a service is listening; closed means that no service is listening; filtered means the probe was dropped, usually by a firewall. The -F option scans a smaller set of the 100 most popular ports, and -T4 speeds up timing.



Figure 3. Port-scan output for scanme.nmap.org.

Expected output: 22/tcp open ssh and 80/tcp open http, with the other 98 ports filtered. Other ports may be displayed in your own lab.

Function 3. Identify service versions

Step 1: Use the ports that were found open on a valid target.

Step 2: Run:

```
nmap -sV -p 22,80 scanme.nmap.org
```

Step 3: Check the VERSION column. The -sV option probes services to guess the product and version; the listed ports should be open for useful results. Check versions before making a vulnerability claim.

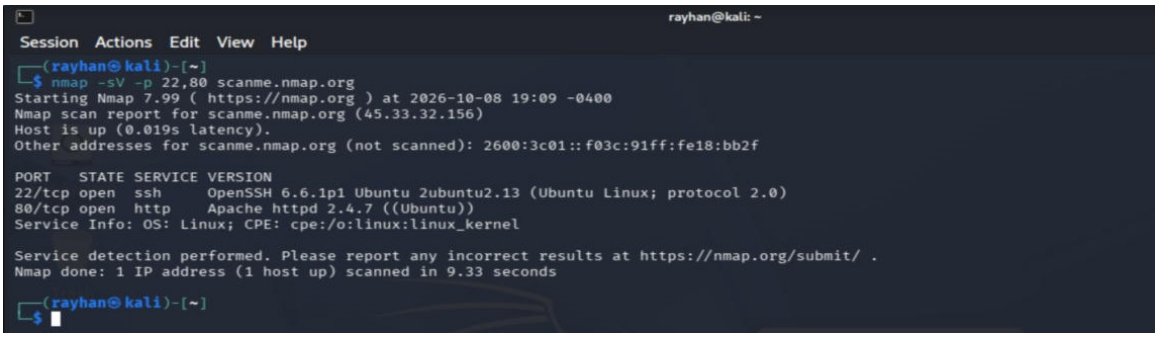


Figure 4. Version-detection output for scanme.nmap.org.

Result interpretation: The VERSION field can contain the product name, the version number, a hint about the operating system, or just a generic protocol name. Here it shows OpenSSH 6.6.1p1 on port 22 and Apache httpd 2.4.7 on port 80. Don't consider missing versions as proof of the security of a service. Document the exact service banner and confirm it, if possible, with administrative access.

Function 4. Run default scripts and save evidence

Step 1: Set up an output folder and run the scan with these commands:

```
mkdir -p scans
nmap -sC -sV -p 22,80 -oA scans/scanme_baseline scanme.nmap.org
```

Step 2: Examine the output files using ls -l scans. The -sC flag runs the default NSE scripts, and the -oA flag saves output in the .nmap, .xml and .gnmap formats. Review the scripts and obtain the appropriate permission before running them.



Figure 5. NSE output and saved files.

3.4 Troubleshooting

Common problems: ‘command not found’ means install Nmap or correct the PATH; ‘requires root privileges’ means use sudo for privileged modes or switch to -sT; ‘host seems down’ means verify addressing and reachability or, within scope, consider -Pn; ‘all ports filtered’ means check the firewall and network mode; DNS errors mean use a validated IP address. See the Nmap Project documentation for details (Nmap Project, n.d.-b).

Problem	Likely cause / response
nmap: command not found	Install Nmap or correct PATH; open a new terminal.
Host seems down	Check the address, VM and firewall; use -Pn only if the host is known to be online and in scope.
Permission denied / root required	Use an authorized administrator session or a nonprivileged scan such as -sT.
All ports filtered	Check target, network path and firewall rules; do not infer that services are absent.
Slow or stalled scan	Reduce the port range and inspect progress; avoid aggressive timing on fragile systems.
Unreliable OS identification	NAT/firewalls can distort fingerprints; corroborate with other evidence.

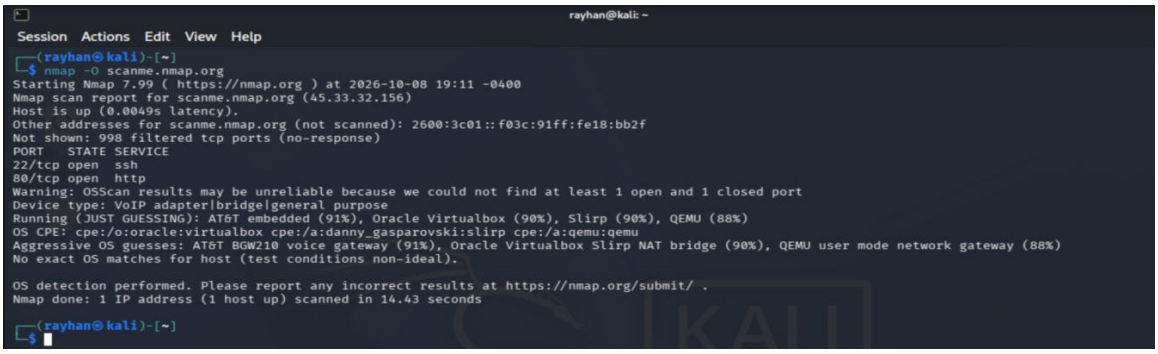


Figure 6. Unreliable OS detection through NAT in my lab.

3.5 Best practices and professional recommendations

Ethics: Do not test beyond scope; get written permission; note the IP range and approved scope of scans. Keep results confidential and share them only with authorized people.

Security: Use isolated VMs for practice, avoid overly aggressive probes and intrusive scripts on production networks, keep tools updated and secure scan output.

Reporting: Log the command, date, target, scan context and the results observed. Identify and validate high-impact issues before making suggestions for repair; differentiate between confirmed issues and suspected weaknesses.

Documentation: Use descriptive names for commands and output files, and keep commands and outputs unabbreviated. Do not include copyrighted material, personal data or sensitive IP address inventories in reports that are made public.

References

Aircrack-ng. (n.d.). Downloads. https://www.aircrack-ng.org/downloads.html

Autopsy. (n.d.). Download. https://www.autopsy.com/download/

Greenbone. (n.d.). Community Edition. https://www.greenbone.net/en/community-edition/

Lyon, G. F. (2009). Nmap network scanning. https://nmap.org/book/

Maltego. (n.d.). Downloads. https://www.maltego.com/downloads/

National Security Agency. (n.d.). Ghidra. https://ghidra-sre.org/

Nmap Project. (n.d.-a). Nmap download. https://nmap.org/download.html

Nmap Project. (n.d.-b). Nmap reference guide. https://nmap.org/book/man.html

Openwall. (n.d.). John the Ripper. https://www.openwall.com/john/

PortSwigger. (n.d.). Burp Suite Community Edition. https://portswigger.net/burp/communitydownload

Rapid7. (n.d.). Metasploit Framework. https://www.metasploit.com/download

Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2008). Technical guide to information security testing and assessment (NIST SP 800-115). https://doi.org/10.6028/NIST.SP.800-115

Security Onion Solutions. (n.d.). Security Onion. https://securityonion.net/

Sullo, C. (n.d.). Nikto [Source code]. https://github.com/sullo/nikto

Wireshark Foundation. (n.d.). Download. https://www.wireshark.org/download.html